A COOKBOOK FOR BACKEND DEVELOPERS

Background Jobs & Queues Cookbook

Move slow work off the request path and run it reliably — with BullMQ (Node) and Celery / RQ (Python) side by side.

BullMQ · Celery · RQ · Redis · Idempotency · Retries & DLQ · Scheduling · Horizon/Flower

---

Written for engineers who know Node.js and are working in Python too, building on Redis as the broker.
Every pattern answers: what it solves, how in Node and Python, the failure modes, and when NOT to use a queue.
Edition 2026 · producers · workers · reliability

Table of Contents

How to Use This Book 4
The one idea: producer, broker, worker 4
Why this matters more than it looks. 5

Part 1 · Why Queues 6
1.1 The problem: slow work on the request path 6
1.2 What belongs on a queue 6
1.3 The shift you have to internalize 7

Part 2 · The Landscape 8
2.1 The libraries 8
2.2 The brokers: Redis vs RabbitMQ vs cloud 8
2.3 Delivery semantics: the fine print 9

Part 3 · Your First Job 10
3.1 BullMQ (Node) 10
3.2 Celery (Python) 10
3.3 RQ (Python) — the simple option 11
3.4 The mapping. 11

Part 4 · Job Options: Delays, Priority, Attempts, Backoff 13
4.1 Delays — run later 13
4.2 Attempts and backoff — retry on failure 13
4.3 Priority — important jobs first 14
4.4 Other useful options 14

Part 5 · Idempotency — the Most Important Chapter 16
5.1 Why a job runs twice 16
5.2 The idempotency toolkit 16
5.3 The order of operations matters 17

Part 6 · Retries, Failures & Dead-Letter Queues 19
6.1 Retry the transient, fail fast on the permanent 19
6.2 The dead-letter queue (DLQ) 20
6.3 Poison messages 20

Part 7 · Flows: Chains, Groups & Dependencies 22
7.1 The three shapes 22
7.2 Chains — sequential steps. 22
7.3 Groups and chords — fan-out then join 22
7.4 Keep orchestration honest. 23

Part 8 · Scheduling & Repeatable Jobs 24
8.1 Repeatable jobs 24
8.2 Why not just use system cron? 24

Part 9 · Concurrency, Rate Limiting & Scaling Workers 26
9.1 Concurrency: jobs in parallel per worker. 26
9.2 Rate limiting: don't overwhelm downstreams 26

9.3 Scaling: more workers, and by queue 27

Part 10 · Monitoring & Observability 29

10.1 The dashboards 29

10.2 The signals that matter 29

10.3 Structured logging and tracing 30

Part 11 · Running Queues in Production 31

11.1 Workers are long-lived processes you must manage 31

11.2 The deploy trap: workers hold old code 31

11.3 Graceful shutdown 31

11.4 Configuration and safety 32

Part 12 · Capstone & Cheat Sheet 33

12.1 Capstone: a notification & webhook pipeline 33

12.2 BullMQ ↔ Celery ↔ RQ cheat sheet 34

12.3 The reliability checklist 34

How to Use This Book

Every backend eventually needs to do work that shouldn't happen inside a web request: send an email, call a slow third-party API, generate a PDF, process an upload, sync a CRM. Doing that work inline makes requests slow and fragile. **Queues** are the answer — you hand the work to a queue and return immediately, and separate **worker** processes do it in the background, with retries and monitoring.

This book teaches queues for engineers who know **Node.js** and are also working in **Python**, so every pattern appears twice: **BullMQ** (the standard Redis queue for Node) and **Celery / RQ** (the standard for Python). Redis is the broker throughout — if you've read the Redis cookbook, this is where Parts 6–9 of it pay off.

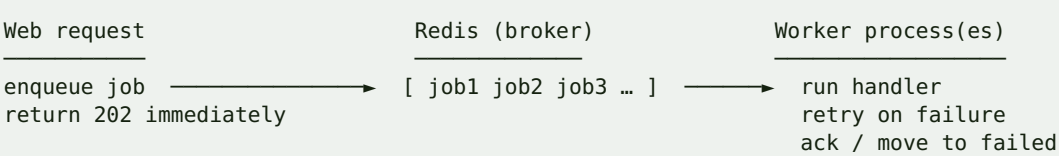
Each chapter answers four questions:

1. **What problem does this solve?**
2. **How do I do it in BullMQ and in Celery/RQ?**
3. **What are the failure modes?**
4. **When should I *not* reach for a queue?**

The one idea: producer, broker, worker

Every queue system is the same three-part shape. A **producer** (your web app) pushes a **job** describing work to do. A **broker** (Redis) holds the jobs. One or more **worker** processes pull jobs and run them, then acknowledge success or record failure.

DIAGRAM



The web request's job is to *enqueue and return*; the worker's job is to *run reliably*. Keeping that split clean is 80% of using queues well.

> **NODE.JS MENTAL MODEL**
>
> If you've used BullMQ, you know this shape already: `queue.add()` is the producer, `new Worker()` is the consumer, Redis is the broker. Celery is the same picture with Python vocabulary: `task.delay()` enqueues, `celery worker` consumes. The concepts transfer one-to-one; only the API names change, and this book maps them the whole way through.

Why this matters more than it looks

Queues seem simple — "just run it later" — but the moment work runs asynchronously you inherit distributed-systems problems: a job might run twice, or fail halfway, or pile up faster than workers drain it, or silently die on deploy. The chapters on **idempotency** (Part 5), **retries and dead-letter queues** (Part 6), and **production** (Part 11) are where the real engineering is. Get those right and queues are a superpower; ignore them and they're a source of duplicate emails and lost work.

By the end you can design and run a production job system: responsive endpoints that offload work, jobs that are safe to retry, sensible failure handling, scheduling, monitoring, and a clean deploy story — in both Node and Python.

Part 1 · Why Queues

Before the how, be clear on the why — and on when a queue is the wrong tool. This part frames the problem queues solve and the mental shift from synchronous to asynchronous work.

1.1 The problem: slow work on the request path

A web request should be fast. When it does slow work inline — sending mail, calling an external API, resizing images — three bad things happen: the user waits, a worker/process is tied up (reducing throughput), and any failure in that slow work becomes a failed request.

JAVASCRIPT / NODE.JS

```
// BAD: the user waits for the email, and a mail outage fails the signup
app.post("/signup", async (req, res) => {
  const user = await createUser(req.body);
  await sendWelcomeEmail(user);        // slow + can fail + not needed for the
response
  await syncToCRM(user);               // even slower, third-party
  res.status(201).json(user);
});
```

JAVASCRIPT / NODE.JS

```
// GOOD: enqueue the slow work and respond immediately
app.post("/signup", async (req, res) => {
  const user = await createUser(user);
  await emailQueue.add("welcome", { userId: user.id });   // returns in ~1ms
  await crmQueue.add("sync", { userId: user.id });
  res.status(201).json(user);          // fast, and resilient to mail/CRM outages
});
```

The endpoint now returns in milliseconds and no longer fails when the mail provider hiccups — the job simply retries on a worker.

1.2 What belongs on a queue

Put on a queue	Keep inline
Sending email / SMS / push	Reading data needed for the response
Calling slow or flaky third-party APIs	Writing the record the user just created
Image/video/PDF processing	Validation and authorization
Data exports, report generation	Anything the response body depends on

Put on a queue	Keep inline
Webhooks you *send*	The primary write in a transaction
CRM/analytics sync	Cheap, must-be-consistent updates
Anything retryable and not needed for the response	

The test: **does the HTTP response depend on this finishing?** If no, queue it. If yes, do it inline.

1.3 The shift you have to internalize

Moving work to a queue changes its guarantees. Inline code runs **once**, **now**, and you see the result. A queued job runs **later**, **maybe more than once**, on **another machine**, and the original request is long gone when it does. That's the price of decoupling, and it drives everything: jobs must be **idempotent** (safe to run twice — Part 5), **serializable** (their input must survive a trip through Redis), and **observable** (you need dashboards because there's no request to watch).

COMMON MISTAKE

Passing a big object or a live handle (a DB connection, a request object, a Node stream) as job data. Job payloads are serialized to JSON/bytes and stored in Redis, then rehydrated later on a different process — so pass **ids and primitives**, and re-fetch fresh data inside the job. Passing the whole user object also means the job runs on *stale* data captured at enqueue time.

WHEN NOT TO USE A QUEUE

A queue adds real complexity: another process to run, deploy, and monitor; serialization; retries; idempotency; a dashboard. Don't add it for work that's already fast and reliable, or for something the response genuinely needs. For a single fire-and-forget task in a small app you can sometimes start with the framework's lightweight background mechanism (FastAPI `BackgroundTasks`, a `setImmediate`) — but the moment the work matters, retries, or must survive a restart, move it to a real queue.

EXERCISE 1.1

Take an existing endpoint that does two things inline (e.g. create a record + send a notification). Split it: keep the create inline, move the notification to a queue that just logs "would send to user N". Measure the endpoint's response time before and after, and write one sentence on why the endpoint is now also more *reliable*, not just faster.

Part 2 · The Landscape

Before writing a job, know your options — the libraries in each language and the brokers underneath them. Picking well up front saves a painful migration later.

2.1 The libraries

Library	Language	Broker	Notes
BullMQ	Node	Redis	The modern standard; rich features (flows, rate limit, repeatable)
Bee-Queue	Node	Redis	Lighter, fast, fewer features
Celery	Python	Redis / RabbitMQ	The heavyweight standard; scheduling, chains, huge ecosystem
RQ	Python	Redis	Simple, Pythonic, minimal — great for straightforward needs
Dramatiq	Python	Redis / RabbitMQ	Simpler than Celery, good defaults
arq	Python	Redis	async-native (asyncio) — fits FastAPI

For this book: **BullMQ** on the Node side, and **Celery** (with **RQ** shown for the simple cases) on the Python side. If your Python app is fully async (FastAPI), **arq** is worth a look as an asyncio-native alternative.

2.2 The brokers: Redis vs RabbitMQ vs cloud

The **broker** is where jobs live between enqueue and execution. Your library usually dictates or recommends one, but the trade-offs are worth knowing.

Broker	Model	Strengths	Watch out for
Redis	in-memory data store used as a queue	fast, simple, you likely already run it	durability is best-effort (Part 10 of the Redis book)
RabbitMQ	dedicated message broker (AMQP)	strong routing, acks, durability	another system to run and learn
SQS (AWS)	managed cloud queue	zero-ops, scalable, durable	higher latency, at-least-once, vendor lock
Kafka	distributed log	huge throughput, replay	overkill for task queues; it's a log, not a job queue

NODE.JS MENTAL MODEL

BullMQ + Redis is the Node default and maps exactly onto Celery/RQ + Redis in Python — same broker, same shape, different client. If you already run Redis for caching (you probably do), you already have a queue broker. RabbitMQ and SQS enter the picture when you need their specific guarantees (advanced routing, managed durability), not by default.

2.3 Delivery semantics: the fine print

Every queue makes a delivery promise, and it's almost never "exactly once":

- **At-least-once** (Redis/BullMQ/Celery/SQS default): a job runs *one or more* times — a worker crash after doing the work but before acking causes a redelivery. **This is why jobs must be idempotent** (Part 5).
- **At-most-once**: a job runs zero or one time — no redelivery, so a crash can *lose* work. Rarely what you want for important jobs.
- **Exactly-once**: the holy grail, and effectively impossible to guarantee end-to-end in a distributed system. You *approximate* it with at-least-once delivery + idempotent handlers.

COMMON MISTAKE

Assuming your queue delivers exactly once and writing jobs that break if run twice (double-charging a card, sending two emails). Every mainstream queue is at-least-once. Design every job to be safe to run again (Part 5) — that, not the broker's marketing, is what makes your system correct.

WHEN NOT TO REACH FOR THE HEAVYWEIGHT

Don't start with Celery's full machinery (broker + result backend + Beat + Flower) for one background task — **RQ** or **arq** (Python) or a lean BullMQ setup is far less to run and understand. Scale up to Celery when you need its scheduling, routing, and workflow features. Match the tool to the job count and reliability you actually need, not the most powerful option.

EXERCISE 2.1

Write a two-column decision note for your own project: which library (BullMQ / Celery / RQ / arq) and which broker (Redis / RabbitMQ / SQS) you'd choose, and *why*, given your stack, existing infra, and reliability needs. Name the one guarantee (at-least-once) that shapes how you'll write every job.

Part 3 · Your First Job

Time to build the full producer → broker → worker loop in both stacks. Same job (send a welcome email), shown in BullMQ and in Celery, with RQ for contrast.

3.1 BullMQ (Node)

A **Queue** enqueues; a **Worker** processes. They connect to the same Redis and the same queue name.

JAVASCRIPT / NODE.JS

```
// queue.js — the producer side (imported by your web app)
import { Queue } from "bullmq";
const connection = { host: "127.0.0.1", port: 6379 };
export const emailQueue = new Queue("email", { connection });

// enqueue from a request handler
await emailQueue.add("welcome", { userId: 42 });     // name + data
```

JAVASCRIPT / NODE.JS

```
// worker.js — a SEPARATE process you run alongside the web app
import { Worker } from "bullmq";
const connection = { host: "127.0.0.1", port: 6379 };

const worker = new Worker("email", async (job) => {
  if (job.name === "welcome") {
    const user = await db.user.findUnique({ where: { id: job.data.userId } });
    await mailer.sendWelcome(user);                  // the actual work
  }
}, { connection, concurrency: 5 });

worker.on("completed", (job) => logger.info(`done ${job.id}`));
worker.on("failed", (job, err) => logger.error(`failed ${job?.id}: ${err.message}`));
```

Run the worker as its own process: `node worker.js`.

3.2 Celery (Python)

A task is a decorated function; the same module is imported by the web app (to enqueue) and run by the worker (to execute).

PYTHON

```python
# tasks.py
from celery import Celery
app = Celery("myapp", broker="redis://localhost:6379/0",
             backend="redis://localhost:6379/1")

@app.task
def send_welcome(user_id: int):
    user = get_user(user_id)
    mailer.send_welcome(user)          # the actual work

# enqueue from a request handler
send_welcome.delay(42)                 # .delay(args) == apply_async
```

Run the worker: `celery -A tasks worker --loglevel=info --concurrency=5`.

3.3 RQ (Python) — the simple option

RQ enqueues a plain function reference; no decorators, no broker/back-end config beyond Redis.

PYTHON

```python
# enqueue
from redis import Redis
from rq import Queue
q = Queue("email", connection=Redis())
q.enqueue(send_welcome, 42)            # function + args

# worker (CLI): rq worker email
```

3.4 The mapping

Concept	BullMQ (Node)	Celery (Python)	RQ (Python)
Define work	`new Worker(name, fn)`	`@app.task def f()`	a plain function
Enqueue	`queue.add(name, data)`	`f.delay(args)`	`q.enqueue(f, args)`
Run workers	`node worker.js`	`celery -A app worker`	`rq worker <queue>`
Broker	Redis	Redis / RabbitMQ	Redis
Payload	JSON `data`	function args (pickled/JSON)	function args
Result	via events / QueueEvents	result backend	job.result

NODE.JS MENTAL MODEL

BullMQ's `queue.add(name, data)` / `new Worker(name, fn)` is the shape in your head already. Celery's twist is that the *task function itself* is both the definition and the thing you call `.delay()` on — the decorator registers it so the worker (which imports the same module) knows how to run it. RQ is the closest to "just run this function later," which is why it's the gentlest starting point.

COMMON MISTAKE

Forgetting that the **worker is a separate process** you must actually run (and keep running). Enqueuing works fine from the web app, jobs pile up in Redis, and nothing happens — because no worker is consuming. In dev you run the worker in a second terminal; in production it's its own service/container (Part 11). "Jobs aren't running" is nearly always "no worker is up" or "the worker points at a different queue/Redis."

EXERCISE 3.1

Build the full loop in your primary language: a queue, an enqueue from a tiny HTTP endpoint, and a worker in a separate process that logs the job. Enqueue 5 jobs, start the worker, watch them drain. Then stop the worker, enqueue 5 more, and confirm they wait in Redis until you start it again. Port the same to the other language.

Part 4 · Job Options: Delays, Priority, Attempts, Backoff

A job is more than a function call — it carries options that control *when* it runs, *how often* it's retried, and in *what order*. These options are where a lot of production behavior lives.

4.1 Delays — run later

Schedule a job for the future: a reminder in 24 hours, a retry of a failed webhook in 5 minutes.

JAVASCRIPT / NODE.JS

```
// BullMQ — delay in milliseconds
await emailQueue.add("reminder", { userId: 42 }, { delay: 24 * 60 * 60 * 1000 });
```

PYTHON

```
# Celery — countdown (seconds) or eta (datetime)
send_reminder.apply_async(args=[42], countdown=86400)
send_reminder.apply_async(args=[42], eta=datetime(2026, 1, 1, 9, 0))
```

4.2 Attempts and backoff — retry on failure

If a job throws, the queue can retry it automatically. You set the number of attempts and the **backoff** (how long to wait between tries) — exponential backoff with jitter is the production default, so retries don't stampede a struggling dependency.

JAVASCRIPT / NODE.JS

```
// BullMQ
await emailQueue.add("welcome", { userId: 42 }, {
  attempts: 5,
  backoff: { type: "exponential", delay: 1000 },   // 1s, 2s, 4s, 8s, 16s
});
```

PYTHON

```
# Celery — retry with exponential backoff + jitter
@app.task(bind=True, max_retries=5, autoretry_for=(Exception,),
          retry_backoff=True, retry_jitter=True)
def send_welcome(self, user_id):
    user = get_user(user_id)
    mailer.send_welcome(user)
```

PRODUCTION TIP

Always cap attempts and always use **exponential backoff with jitter** for anything that hits an external service. A fixed short retry hammers a down dependency and can turn a blip into an outage (a "retry storm"); jitter spreads retries out so they don't synchronize. Match `attempts` to how transient the failure is: a flaky network call deserves several retries, a validation error deserves zero (it'll never succeed — fail fast).

4.3 Priority — important jobs first

When jobs of different urgency share a queue, priority (or separate queues) lets urgent work jump ahead.

JAVASCRIPT / NODE.JS

```
await emailQueue.add("password-reset", data, { priority: 1 });   // lower = sooner
await emailQueue.add("newsletter", data, { priority: 10 });
```

PYTHON

```
# Celery: prefer separate named queues + routing over numeric priority
send_reset.apply_async(args=[uid], queue="high")
send_news.apply_async(args=[uid], queue="low")
```

WHEN NOT TO LEAN ON NUMERIC PRIORITY

Numeric priority within one queue helps a little, but the more robust pattern is **separate queues by urgency** (`critical` , `default` , `bulk`) with workers assigned accordingly (Part 9) — a giant newsletter batch then can't delay a password-reset email even if priorities are set, because they're drained by different workers. Reach for dedicated queues before you fiddle with priority numbers.

4.4 Other useful options

Option	BullMQ	Celery	Purpose
Delay	`delay: ms`	`countdown` / `eta`	run later
Attempts	`attempts: n`	`max_retries`	retry count
Backoff	`backoff: {...}`	`retry_backoff`	spacing between retries
Priority	`priority: n`	queue routing	ordering
Timeout	`job timeout` opt	`time_limit` / `soft_time_limit`	kill a stuck job
Remove on complete	`removeOnComplete`	result expiry	keep Redis lean
Dedupe / unique	`jobId`	custom / locks	avoid duplicates (Part 5)
Rate limit	worker `limiter`	`rate_limit`	throttle throughput (Part 9)

COMMON MISTAKE

Leaving completed jobs in Redis forever. By default some setups keep every finished job, and Redis memory creeps up until it evicts or OOMs. Set `removeOnComplete` (keep, say, the last 1000) and `removeOnFail` policies in BullMQ, or result-expiry in Celery, so finished jobs are trimmed. Keep enough history to debug, not all of it forever.

EXERCISE 4.1

Give a job `attempts: 5` with exponential backoff and make it fail the first 3 times (throw unless `attemptsMade >= 3`); watch it retry with growing delays and eventually succeed. Add a second, delayed job (run in 10s) and a high-priority job, and confirm ordering. Set `removeOnComplete` and verify finished jobs don't accumulate in Redis (`LLEN` / `ZCARD` the relevant keys).

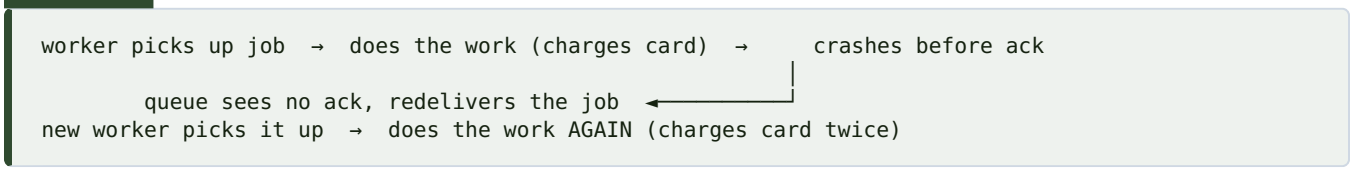
Part 5 · Idempotency — the Most Important Chapter

If you remember one thing from this book, remember this: **jobs run at least once, which means sometimes more than once — so every job must be safe to run twice.** A job that isn't idempotent will eventually double-charge a customer, send duplicate emails, or corrupt a counter, because a worker *will* crash after doing the work but before acknowledging it, and the queue *will* redeliver.

5.1 Why a job runs twice

The redelivery happens through no fault of your code:

DIAGRAM



Network blips, deploys that kill workers mid-job, timeouts, and manual retries all cause the same thing. You cannot prevent redelivery; you can only make it harmless.

5.2 The idempotency toolkit

1. Guard on state you can check. Before doing the effect, check whether it's already done.

JAVASCRIPT / NODE.JS

```
async function chargeOrder(job) {
  const order = await db.order.findUnique({ where: { id: job.data.orderId } });
  if (order.status === "charged") return;            // already done — no-op
  const chargeId = await gateway.charge(order.totalCents);
  await db.order.update({ where: { id: order.id }, data: { status: "charged",
chargeId } });
}
```

2. Use an idempotency key with the external service. Payment and email APIs accept an `Idempotency-Key`; the same key never performs the action twice, even if you call them twice.

PYTHON

```
@app.task(bind=True, max_retries=5)
def charge_order(self, order_id):
```

```
    order = get_order(order_id)
    if order.status == "charged":
        return
    charge = gateway.charge(order.total_cents,
                            idempotency_key=f"order-{order_id}")   # provider dedupes
    mark_charged(order_id, charge.id)
```

3. Make the write itself idempotent. A unique DB constraint (`UNIQUE(order_id)` on a `payments` table) turns a duplicate into a caught error instead of a second charge. `INSERT ... ON CONFLICT DO NOTHING` / `updateOrCreate` are your friends.

4. Deduplicate at enqueue. Give the job a stable `jobId` (BullMQ) so enqueuing "the same" job twice collapses to one — useful for "sync user 42" that might be triggered repeatedly.

JAVASCRIPT / NODE.JS

```
await queue.add("sync", { userId: 42 }, { jobId: `sync-user-42` });  // dedupes
```

Technique	Guards against	Where
State check (`if already done`)	re-running the effect	in the handler
Provider idempotency key	duplicate external calls	at the API boundary
Unique DB constraint	duplicate rows/effects	in the database
Stable `jobId` dedupe	enqueuing duplicates	at enqueue time

5.3 The order of operations matters

Do the **external effect** first, then record success — and make the recording itself the idempotency guard. If you record success *before* doing the effect and then crash, you'll skip real work; if you do the effect and crash before recording, the state check must catch the redelivery. When the effect and the record can't be one atomic step, lean on the provider's idempotency key so the *effect* is safe regardless of ordering.

COMMON MISTAKE

Writing a "send email" job with no guard, then watching users get 3 welcome emails after a deploy restarts workers mid-batch. Every job that has a side effect — sends, charges, writes, external calls — needs at least one idempotency technique above. Treat "what happens if this runs twice?" as a required design question for every single job, not an edge case.

PRODUCTION TIP

The cheapest, most general guard is a **unique idempotency key stored in your database**: at the start of the job, `INSERT` a row keyed by a natural id (`order-42-charge`); if the insert conflicts, the job already ran — return. This works even across queue systems and provider quirks, and it's the same pattern you use for inbound webhooks (see the Redis and framework cookbooks).

EXERCISE 5.1

Write a `charge_order` job and deliberately run it twice for the same order (simulate redelivery by calling the handler twice). First observe the double charge with no guard, then add a state check *and* a unique constraint on a `payments(order_id)` table, and confirm the second run is a safe no-op. Do it in Node and Python.

Part 6 · Retries, Failures & Dead-Letter Queues

Jobs fail. Networks drop, APIs 500, data is bad. A production queue needs a deliberate answer to "what happens when a job fails" — retry the transient ones, give up on the permanent ones, and never silently lose the failures.

6.1 Retry the transient, fail fast on the permanent

Not all failures are equal. A timeout calling Stripe should retry; a `ValidationError` on malformed data never will — retrying just wastes attempts and delays the inevitable. Distinguish them.

JAVASCRIPT / NODE.JS

```
// BullMQ worker — throw to retry; throw an UnrecoverableError to stop retrying
import { UnrecoverableError } from "bullmq";

const worker = new Worker("email", async (job) => {
  const user = await db.user.findUnique({ where: { id: job.data.userId } });
  if (!user) throw new UnrecoverableError("user gone — don't retry");   // permanent
  await
mailer.sendWelcome(user);                                              // may throw ->
retry
}, { connection, attempts: 5, backoff: { type: "exponential", delay: 1000 } });
```

PYTHON

```
# Celery — retry transient, don't retry permanent
@app.task(bind=True, max_retries=5, retry_backoff=True, retry_jitter=True)
def send_welcome(self, user_id):
    user = get_user(user_id)
    if user is None:
        return                                  # permanent — give up quietly
    try:
        mailer.send_welcome(user)
    except TransientMailError as exc:
        raise self.retry(exc=exc)               # transient — retry with backoff
```

6.2 The dead-letter queue (DLQ)

When a job exhausts its retries, it must go **somewhere you can see and act on** — not vanish. That place is the **dead-letter queue** (or "failed" set): a holding area for jobs that gave up, so you can inspect the error, fix the cause, and replay them.

- **BullMQ** keeps failed jobs in a *failed* set (bounded by `removeOnFail`); you list, inspect (`job.failedReason`, `job.stacktrace`), and `job.retry()` them — Bull Board (Part 10) gives a UI. For a true separate DLQ, move exhausted jobs to a dedicated `dead` queue in your failure handler.
- **Celery** exhausted tasks land in the result backend as failures; teams add a dead-letter routing (or a custom `on_failure` that records them) and Flower to view them.

JAVASCRIPT / NODE.JS

```
// route exhausted jobs to an explicit dead-letter queue for humans to review
worker.on("failed", async (job, err) => {
  if (job.attemptsMade >= job.opts.attempts) {
    await deadQueue.add("dead", { original: job.data, error: err.message, queue:
job.queueName });
    alerting.notify(`job ${job.id} dead-lettered: ${err.message}`);
  }
});
```

6.3 Poison messages

A **poison message** is a job that fails forever — bad data, a bug, an object that no longer exists. Without a retry cap it retries endlessly, burning workers and filling logs. The cap + DLQ combination is exactly what contains it: it fails `attempts` times, then parks in the DLQ instead of looping.

COMMON MISTAKE

Infinite or unbounded retries. A job that always fails and retries forever is a slow-motion outage: it consumes a worker slot on every cycle, and a burst of them starves real work. **Always cap attempts**, always send exhausted jobs to a DLQ, and alert when the DLQ grows — a rising DLQ is your earliest signal that a dependency is down or a deploy shipped a bug.

PRODUCTION TIP

Treat the DLQ as an operational surface, not a graveyard. Monitor its size (alert past a threshold), include enough context in each dead-lettered entry to diagnose (original payload, error, stack, timestamp, attempt count), and build a one-command **replay** path so that once you've fixed the root cause you can requeue the parked jobs. A DLQ you never look at is just a slower way to lose data.

WHEN NOT TO RETRY AT ALL

Some jobs shouldn't retry: anything whose failure is deterministic (validation, a 400 from an API, a missing referenced record) will fail identically every time — retrying wastes resources and delays the alert. Detect those, fail fast (zero retries), and dead-letter immediately so a human sees them. Retries are for *transient* failures only.

EXERCISE 6.1

Build a job that fails transiently (throw on the first 2 attempts, succeed on the 3rd) and one that fails permanently (bad data — never retry). Configure 5 attempts + backoff. Route exhausted jobs to a `dead` queue with the error and payload, and write a small script that lists and replays the DLQ. Trigger a poison message and confirm it lands in the DLQ instead of looping forever.

Part 7 · Flows: Chains, Groups & Dependencies

Real work is often multi-step: charge the order, *then* email a receipt, *then* notify the warehouse; or process 1,000 images in parallel and run a summary when all finish. Queue libraries provide **workflow primitives** so you don't hand- roll this orchestration.

7.1 The three shapes

Shape	Meaning	BullMQ	Celery
Chain	run steps in sequence; stop if one fails	`FlowProducer` (parent/ children)	`chain(a.s(), b.s())`
Group	run many in parallel	multiple `add` s	`group(...)`
Chord	parallel work, then a callback when all finish	Flow with a parent	`chord(group) (callback)`

7.2 Chains — sequential steps

PYTHON

```python
# Celery — each task's result feeds the next; the chain stops on failure
from celery import chain
chain(charge_order.s(order_id), send_receipt.s(), notify_warehouse.s())()
```

JAVASCRIPT / NODE.JS

```javascript
// BullMQ — a FlowProducer builds a parent job with ordered children
import { FlowProducer } from "bullmq";
const flow = new FlowProducer({ connection });
await flow.add({
  name: "checkout", queue: "orders", data: { orderId },
  children: [
    { name: "charge", queue: "orders", data: { orderId } },
    { name: "receipt", queue: "emails", data: { orderId } },
  ],
});   // children complete before the parent runs
```

7.3 Groups and chords — fan-out then join

PYTHON

```python
# Celery — process everything in parallel, then run finalize once, with all results
from celery import group, chord
chord(
    (process_image.s(img_id) for img_id in image_ids),   # parallel group
    build_album.s()                                       # callback after all done
)()
```

This is the pattern for "resize 500 images, then mark the gallery ready" or "call 10 shipping providers, then pick the cheapest."

7.4 Keep orchestration honest

WHEN NOT TO BUILD A BIG WORKFLOW

Flows are powerful and seductive — resist encoding complex branching business logic as deeply nested chains and chords. They're hard to debug (the logic is smeared across task boundaries and a broker), hard to test, and hard to observe. For anything with real conditional logic, prefer a single job that *calls* the steps in plain code (and enqueues follow-ups where you genuinely want them decoupled). Reserve chains/chords for simple, linear "A then B then C" and clean "fan-out then join" — not as a general programming model.

COMMON MISTAKE

Assuming a chain is transactional — it is **not**. If step 2 of a 3-step chain fails after step 1 already ran, step 1's effects are *not* rolled back. Design each step to be idempotent (Part 5) and, where partial completion is dangerous, add compensation (a step that undoes the previous effect) or make the whole operation reconstructable. A chain sequences work; it doesn't give you database-style atomicity across steps.

PRODUCTION TIP

A common, robust alternative to a long chain is a **single orchestrating job** that runs the steps in normal code and only enqueues *separately* decoupled work (the slow, independently-retryable bits). You get readable, testable control flow with the queue used where it earns its keep, instead of a workflow graph you have to reverse-engineer from a dashboard at 2 a.m.

EXERCISE 7.1

Build a checkout flow three steps long (charge → receipt → notify) as a chain, and make the middle step fail; observe that the first step's effect is *not* undone, then add an idempotency guard so re-running the chain is safe. Then build a fan-out/join: enqueue N "process item" jobs and a finalizer that runs only after all complete (chord in Celery, Flow parent in BullMQ).

Part 8 · Scheduling & Repeatable Jobs

Beyond "run this now-ish," you need "run this every night at 2 a.m." and "run this every 5 minutes." Queue systems provide **repeatable/periodic jobs** so you don't bolt on a separate cron that then has to reach into your app.

8.1 Repeatable jobs

JAVASCRIPT / NODE.JS

```javascript
// BullMQ — a repeatable job defined by a cron pattern or an interval
await reportQueue.add("daily-report", {}, {
  repeat: { pattern: "0 2 * * *" },          // every day at 02:00 (cron)
});
await syncQueue.add("crm-sync", {}, {
  repeat: { every: 5 * 60 * 1000 },          // every 5 minutes
});
```

PYTHON

```python
# Celery Beat — the scheduler process that enqueues periodic tasks
from celery.schedules import crontab
app.conf.beat_schedule = {
    "daily-report": {"task": "tasks.daily_report", "schedule": crontab(hour=2,
minute=0)},
    "crm-sync":     {"task": "tasks.crm_sync",     "schedule": 300.0},   # every 300s
}
# run the scheduler:  celery -A tasks beat
```

The key difference in Python: Celery needs a **separate Beat process** (`celery beat`) that enqueues periodic tasks; the workers still do the running. BullMQ builds repeatable scheduling into the queue itself.

Concept	node-cron / cron	BullMQ	Celery
Where it runs	in a process / OS	in the queue	Beat process
Frequency	cron expression	`pattern` or `every`	`crontab` or seconds
Does the work	inline (risky)	a worker (good)	a worker (good)
Survives restart	depends	yes (in Redis)	yes (schedule is config)

8.2 Why not just use system cron?

You can, but a cron job that does heavy work inline has the same problems Part 1 described — no retries, no visibility, and it competes with your app for resources. The queue-native

approach is better: cron/Beat/repeatable only *enqueues* a job; a worker runs it with retries, backoff, and monitoring like any other job.

PRODUCTION TIP

Let the scheduler **dispatch, not do**. A repeatable job should enqueue the real work (or be a thin task that itself enqueues), so heavy nightly work runs on a normal worker with retries and shows up in your dashboards — not as an opaque cron that either worked or didn't. And in a multi-instance deployment, make sure only **one** scheduler runs (one Beat process; BullMQ dedupes repeatable jobs by key) or you'll fire every scheduled job N times.

COMMON MISTAKE

Running the scheduler on every app instance and firing each periodic job multiple times — the multi-server version of a double-send. Run exactly one Celery Beat; for BullMQ, the repeatable-job key dedupes, but don't also add a separate OS cron doing the same thing. Overlap is the other trap: if a 5-minute job sometimes takes 6 minutes, guard against a second copy starting (a lock, or `maxStalledCount` / `concurrency: 1` on that queue).

WHEN NOT TO SCHEDULE INSIDE THE QUEUE

For truly infrastructure-level schedules (database backups, cert renewal, log rotation) that aren't part of your app's domain, plain system cron or your platform's scheduler is simpler and more appropriate. Use queue-native scheduling for *application* work (reports, digests, syncs, cleanups) that benefits from retries and observability.

EXERCISE 8.1

Schedule a repeatable "cleanup" job every minute (BullMQ `repeat.every` or a Celery Beat entry) that just logs and enqueues a real `do_cleanup` job. Run a worker and confirm it fires on schedule and the real job runs with retries. Then start a second scheduler and observe the double-fire; fix it by ensuring a single scheduler.

Part 9 · Concurrency, Rate Limiting & Scaling Workers

Throughput comes from running jobs in parallel — but more parallelism isn't always more speed, and unbounded parallelism can overwhelm your database or a rate-limited API. This part is about tuning how many jobs run at once, and where.

9.1 Concurrency: jobs in parallel per worker

Each worker process can run several jobs concurrently. In Node that's the event loop juggling I/O-bound jobs; in Python it's worker processes/threads (mind the GIL — see the Python cookbook).

JAVASCRIPT / NODE.JS

```
// BullMQ — one worker process, up to 10 jobs in flight (good for I/O-bound work)
const worker = new Worker("email", handler, { connection, concurrency: 10 });
```

PYTHON

```
# Celery — concurrency = number of worker child processes (prefork)
# celery -A tasks worker --concurrency=8
# for I/O-bound tasks, a gevent/eventlet pool allows far higher concurrency:
# celery -A tasks worker --pool=gevent --concurrency=200
```

NODE.JS MENTAL MODEL

BullMQ concurrency is like how many `await`s you let run at once on one event loop — great for I/O-bound jobs (API calls, DB), useless for CPU-bound ones (a synchronous 100ms crunch blocks the loop). Celery's default *prefork* pool uses separate processes (real CPU parallelism, bypassing the GIL), while its gevent/eventlet pools mirror the Node model for I/O-bound work. Match the pool to the workload, exactly as you would in the runtime chapters.

9.2 Rate limiting: don't overwhelm downstreams

When jobs call a rate-limited API (or a fragile database), cap how fast the queue runs them — independent of how many are waiting.

JAVASCRIPT / NODE.JS

```
// BullMQ — at most 100 jobs per 60s across this worker
const worker = new Worker("sms", handler, {
  connection,
```

```
  limiter: { max: 100, duration: 60_000 },
});
```

PYTHON

```python
# Celery — per-task rate limit
@app.task(rate_limit="100/m")
def send_sms(to, body): ...
```

9.3 Scaling: more workers, and by queue

Two axes of scaling: **up** (more concurrency per worker) and **out** (more worker processes/containers). The important refinement is scaling **per queue**: give latency-sensitive work its own workers so a flood of bulk jobs can't starve it.

DIAGRAM

```
queue: critical  ──►  workers A,B      (password resets, 2FA — never wait)
queue: default   ──►  workers C,D,E    (normal jobs)
queue: bulk      ──►  worker F         (newsletters, exports — can be slow)
```

Lever	BullMQ	Celery
Jobs in parallel per worker	`concurrency`	`--concurrency` + pool type
Throttle rate	`limiter`	`rate_limit`
Scale out	run more worker processes	run more worker processes
Isolate by urgency	separate queues + workers	`-Q queue` per worker
Autoscale	by queue depth (ops)	by queue depth (ops)

PRODUCTION TIP

Route jobs to queues by *urgency and cost*, and give each queue workers sized to its needs: a dedicated worker for the `bulk` queue means a 50,000-email newsletter can't delay a `critical` password-reset. Set concurrency from the workload type (high for I/O-bound, ~CPU-count for CPU-bound) and add a `limiter` / `rate_limit` wherever a job calls something with its own quota. Autoscale worker count on **queue depth**, not CPU.

COMMON MISTAKE

Cranking concurrency up to "go faster" and instead knocking over the database (connection-pool exhaustion) or getting rate-limited/banned by a third-party API. Concurrency is bounded by your *slowest shared resource*, not your worker. Size it to what the downstream can take, add a rate limiter for external calls, and watch DB connections and API 429s as you scale.

WHEN NOT TO ADD MORE CONCURRENCY

If jobs are CPU-bound, piling on concurrency in a single Node worker or a single-threaded pool does nothing (or hurts) — you need more *processes* (BullMQ: more worker processes; Celery: prefork). And if the queue is draining fine and latency is acceptable, don't scale preemptively; idle workers cost money and add deploy surface. Scale on a measured backlog.

EXERCISE 9.1

Create three queues (`critical` , `default` , `bulk`) and workers assigned to each. Flood `bulk` with 10,000 trivial jobs, then enqueue one `critical` job, and confirm it runs promptly because a separate worker serves it. Add a `limiter` / `rate_limit` to a job that calls a mock API and verify it never exceeds the configured rate.

Part 10 · Monitoring & Observability

A queue runs where no user is watching, so if you can't *see* it, you can't operate it. The failure mode of an unmonitored queue is silent: jobs pile up, or fail, or slow down, and you find out from an angry customer. This part is the dashboards and signals that keep that from happening.

10.1 The dashboards

Tool	Stack	Shows
Bull Board / Taskforce	BullMQ	queues, waiting/active/completed/failed, retry & inspect
Flower	Celery	tasks, workers, rates, failures, real-time
Horizon	Laravel (for contrast)	throughput, wait time, failed jobs, auto-balancing
RQ Dashboard	RQ	queues, jobs, failed registry

JAVASCRIPT / NODE.JS

```
// Bull Board — mount a UI in an Express app (dev/admin only)
import { createBullBoard } from "@bull-board/api";
import { BullMQAdapter } from "@bull-board/api/bullMQAdapter";
import { ExpressAdapter } from "@bull-board/express";
const serverAdapter = new ExpressAdapter();
createBullBoard({ queues: [new BullMQAdapter(emailQueue)], serverAdapter });
app.use("/admin/queues", serverAdapter.getRouter());   // protect this route!
```

SHELL

```
# Flower for Celery
celery -A tasks flower --port=5555
```

10.2 The signals that matter

Watch these, and alert on the ones that mean trouble:

Signal	Healthy	Alarm when
Queue depth (waiting jobs)	stable / near zero	growing steadily → workers can't keep up
Failed / DLQ size	low, flat	rising → a dependency is down or a bug shipped
Job latency (enqueue→done)	within SLA	climbing → backlog forming
Processing time per job	steady	spiking → a slow dependency
Worker count / health	matches expected	dropped → workers crashed or didn't deploy

Signal	Healthy	Alarm when
Retry rate	low	high → transient failures piling up

The two you should page on: **queue depth growing without bound** (you're falling behind — scale workers or fix a stuck job) and **DLQ/failed rising** (something is broken — investigate now).

10.3 Structured logging and tracing

Log each job with structured context — job id, queue, attempt, duration, outcome — to the same log pipeline as your app, and carry a **correlation id** from the originating request into the job so you can trace one user action across the HTTP request *and* its background jobs.

JAVASCRIPT / NODE.JS

```
worker.on("completed", (job) => log.info({ jobId: job.id, queue: job.queueName,
  ms: Date.now() - job.processedOn, reqId: job.data.reqId }, "job.completed"));
worker.on("failed", (job, err) => log.error({ jobId: job?.id, attempt:
job?.attemptsMade,
  err: err.message }, "job.failed"));
```

PRODUCTION TIP

Emit **metrics** (queue depth, completed/failed counts, job duration histograms) to Prometheus/Datadog, not just logs — you want graphs and alert thresholds, not grep. Track *latency* (time from enqueue to completion), not only throughput: a queue can be processing fast and still be badly behind if the backlog is growing. And put a correlation id on every job so a support ticket ("my export never arrived") is traceable end to end.

COMMON MISTAKE

Shipping a queue with no dashboard and no alert on queue depth, then discovering during an incident that jobs have been backing up for hours (or that the worker never started after last night's deploy). Stand up Bull Board/Flower and a queue-depth + DLQ alert on **day one** — before you need them. Also: exposing the dashboard publicly — always put it behind auth, it reveals payloads and lets anyone retry/delete jobs.

EXERCISE 10.1

Mount Bull Board (or run Flower) against your dev queue. Enqueue a mix of succeeding and failing jobs and watch the states move. Add structured completed/failed logging with a correlation id passed from an HTTP request into the job. Then simulate a backlog (stop the worker, enqueue 1,000 jobs) and describe what a "queue depth growing" alert would have told you.

Part 11 · Running Queues in Production

The code is the easy part; operating workers reliably is where teams get burned. This part covers the deploy and runtime concerns that don't show up in a tutorial but decide whether your queue survives contact with production.

11.1 Workers are long-lived processes you must manage

A worker is a separate, long-running process. That means it needs the same care as your web service: a process supervisor to restart it, health signals, and a deploy story.

DIAGRAM

```
web service        →  handles requests, enqueues jobs
worker service(s)  →  run jobs   (scale independently)
scheduler          →  Celery Beat / BullMQ repeatable owner (exactly one)
Redis              →  broker
dashboard          →  Bull Board / Flower (behind auth)
```

Run workers under a supervisor (systemd, Docker `restart: unless-stopped`, a Kubernetes Deployment, or a platform worker dyno) so a crash restarts them automatically.

11.2 The deploy trap: workers hold old code

This is the single most common production surprise. A worker imports your code at startup and holds it in memory, so **a running worker keeps executing the old code until you restart it.** Deploying new code without restarting workers means jobs run the previous version — sometimes for hours.

> **COMMON MISTAKE**
>
> Deploying and forgetting to restart workers, so new job logic never takes effect (or worse, old and new code run simultaneously during a rollout, processing the same queue with different behavior). Your deploy pipeline **must** restart the worker processes as a step — every time. This is the exact analog of Laravel's `queue:restart` and the Celery/BullMQ worker restart in the framework cookbooks.

11.3 Graceful shutdown

When you restart or scale down a worker, don't kill it mid-job — let it finish the jobs in flight, stop accepting new ones, then exit. Both libraries support this; wire it to `SIGTERM` so your orchestrator's stop signal is handled cleanly.

JAVASCRIPT / NODE.JS

```
// BullMQ — stop taking new jobs, wait for active ones, then exit
process.on("SIGTERM", async () => {
  await worker.close();     // waits for active jobs to finish (up to a grace period)
  process.exit(0);
});
```

Celery handles `TERM` as a warm shutdown by default (finish current tasks, then stop). Give your orchestrator a **stop grace period** longer than your longest job, or set a job `time_limit` so shutdown can't hang forever.

11.4 Configuration and safety

- **Separate Redis logical DBs** (or instances) for cache vs. queue so a `FLUSHDB` on the cache can't wipe queued jobs, and eviction policy differences (Part 4 of the Redis book) don't silently drop jobs. **Never** run a queue on a Redis with `allkeys-lru` eviction — it will evict your jobs.
- **Set job timeouts** so a stuck job (hung API call) doesn't occupy a worker forever.
- **Bound retained jobs** (`removeOnComplete` / `removeOnFail`, result expiry) so Redis memory stays flat.
- **Secure the dashboard** behind auth; it exposes payloads and controls.

PRODUCTION TIP

The queue's Redis is *stateful infrastructure* — treat it like a database, not a disposable cache. Give it persistence appropriate to how much you'd hate to lose in-flight jobs (AOF `everysec`), `noeviction` (not LRU!), and a replica for HA. Combined with idempotent jobs (Part 5), that gives you a system where a Redis restart or a worker crash costs you a redelivery, not a lost job.

WHEN NOT TO CO-LOCATE

Don't run heavy CPU-bound workers on the same machine as your web service — they'll steal CPU and hurt request latency. Give workers their own instances/containers so you can scale and deploy them independently, and so a runaway job can't take down the web tier. The whole point of a queue is decoupling; keep the runtimes decoupled too.

EXERCISE 11.1

Write a docker-compose with four services: `web`, `worker`, `scheduler` (Beat or a BullMQ repeatable owner), and `redis` (with AOF on, eviction off). Add a graceful-shutdown handler to the worker and confirm that `docker compose stop worker` lets an in-flight job finish. Then simulate a deploy: change the job's log message, rebuild, and prove the worker only picks up the change after a restart.

Part 12 · Capstone & Cheat Sheet

Tie it together with a realistic pipeline, then keep the cheat sheet at your desk.

12.1 Capstone: a notification & webhook pipeline

Build the async backbone of a real app: when something happens (an order is paid), send the customer an email *and* deliver a webhook to their configured endpoint — both off the request path, both reliable.

Architecture

DIAGRAM

```
POST /orders/:id/pay  ──►  charge inline (must be consistent)
                           enqueue:  notify(orderId)      → emails queue
                           enqueue:  deliver_webhook(id)  → webhooks queue
                           return 200 immediately
                                 │
emails worker     ──►  render + send email (idempotent: guard on sent flag)
webhooks worker   ──►  sign payload (HMAC), POST to customer URL,
                       retry with backoff, dead-letter after N attempts
```

The jobs

JAVASCRIPT / NODE.JS

```
// notify job — idempotent email (Part 5)
new Worker("emails", async (job) => {
  const order = await db.order.findUnique({ where: { id: job.data.orderId } });
  if (order.receiptSentAt) return;                          // already sent — no-op
  await mailer.sendReceipt(order);
  await db.order.update({ where: { id: order.id }, data: { receiptSentAt: new
Date() } });
}, { connection, concurrency: 10 });

// webhook delivery — retries, backoff, DLQ (Parts 4, 6)
new Worker("webhooks", async (job) => {
  const { url, secret, payload } = job.data;
  const sig = hmacSha256(secret, JSON.stringify(payload));
  const res = await fetch(url, {
    method: "POST",
    headers: { "X-Signature": sig, "content-type": "application/json" },
    body: JSON.stringify(payload),
    signal: AbortSignal.timeout(10_000),
  });
  if (!res.ok) throw new Error(`endpoint ${res.status}`);  // triggers retry
}, { connection, concurrency: 20, limiter: { max: 200, duration: 1000 } });
// enqueue with: attempts: 8, backoff exponential; on exhaustion → dead queue
```

The Celery version is the same shape: two tasks (`send_receipt`, `deliver_webhook`), `max_retries` + `retry_backoff`, an idempotency guard, and a dead-letter path. Everything you learned appears here: offloading (Part 1), idempotency (Part 5), retries + DLQ (Part 6), rate limiting (Part 9), and monitoring (Part 10).

What to verify

- The endpoint returns fast and stays up even if mail/webhook targets are down.
- Re-running `notify` sends **one** email (idempotency).
- A flaky webhook endpoint (500 then 200) is retried with backoff and eventually delivered; a permanently-dead endpoint lands in the DLQ after N attempts.
- Deploying new job code requires a worker restart to take effect (Part 11).

12.2 BullMQ ↔ Celery ↔ RQ cheat sheet

Task	BullMQ	Celery	RQ
Define work	`new Worker(name, fn)`	`@app.task def f()`	plain function
Enqueue	`queue.add(name, data)`	`f.delay(a)` / `apply_async`	`q.enqueue(f, a)`
Delay	`{ delay: ms }`	`countdown=` / `eta=`	`enqueue_in(td, f)`
Retries	`{ attempts, backoff }`	`max_retries` + `retry_backoff`	`Retry(max=...)`
Priority	`{ priority }` / queues	queue routing	multiple queues
Repeatable	`{ repeat: {pattern} }`	Celery **Beat**	rq-scheduler
Chain	`FlowProducer`	`chain(...)`	dependency `depends_on`
Fan-out + join	Flow parent	`chord(group)(cb)`	—
Rate limit	worker `limiter`	`rate_limit`	—
Concurrency	`{ concurrency }`	`--concurrency` + pool	worker count
Dead-letter	failed set + manual DLQ	failure routing / `on_failure`	failed registry
Dashboard	Bull Board	Flower	RQ Dashboard
Graceful stop	`worker.close()` on SIGTERM	warm shutdown on TERM	warm shutdown

12.3 The reliability checklist

- [ ] Only non-response work is queued; response-critical work stays inline (P1)
- [ ] Job payloads are ids/primitives, not big/live objects (P1)
- [ ] Every job with a side effect is **idempotent** (P5)
- [ ] `attempts` capped + exponential backoff with jitter (P4, P6)
- [ ] Exhausted jobs go to a **DLQ**; DLQ size is alerted (P6)

- [ ] Transient vs permanent failures distinguished (fail fast on permanent) (P6)
- [ ] Queues split by urgency; workers sized per queue (P9)
- [ ] Rate limits on jobs that call quota'd APIs (P9)
- [ ] Dashboard + queue-depth/DLQ alerts, correlation ids (P10)
- [ ] Deploy restarts workers; graceful shutdown on SIGTERM (P11)
- [ ] Queue Redis: persistence on, **eviction off**, separate from cache (P11)

YOU'RE READY

Queues turn a fragile, slow request into a fast one backed by reliable background work — but only if you respect the two hard truths: jobs run more than once (so make them idempotent) and workers are stateful processes (so deploy and monitor them deliberately). Master those and you can build the async backbone of any serious backend, in Node or Python. Pair this with the **Redis** cookbook (the broker) and the **Mailpit** cookbook (test the emails these jobs send) — and the **Backend Infra Sprint** plan sequences all three.